

B.Sc.CSIT 7th Semester

CSC-420: Data Warehousing and Data Mining

Lab 01 :

Implementation of K-means clustering algorithm

Objective:

Write a Python program to implement K-means Clustering algorithm. Generate 1000 2D data points in the range 0-100 randomly. Divide data points into 3 clusters.

Required Theory:

K-means clustering is a popular unsupervised machine learning algorithm used for partitioning a dataset into a predetermined number of clusters. The goal of K-means is to minimize the within-cluster variance, also known as inertia or sum of squared distances from each point in the cluster to the centroid of that cluster. Here's a step-by-step overview of how the algorithm works:

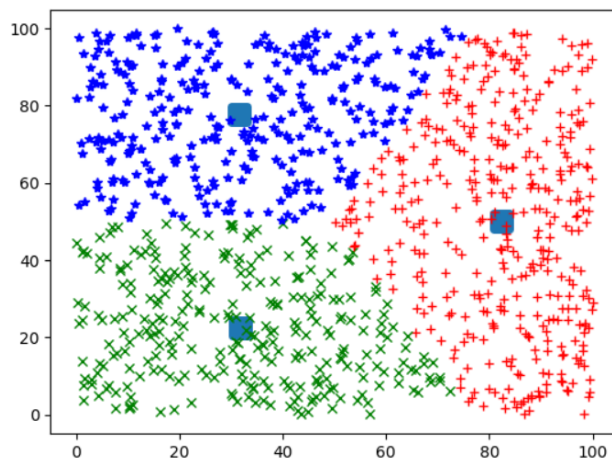
- 1. Initialization:** Choose the number of clusters (K) and randomly initialize K centroids. Centroids are the points that represent the center of each cluster.
- 2. Assignment Step:** Assign each data point to the nearest centroid. This is typically done by calculating the Euclidean distance between each point and each centroid, and assigning each point to the cluster with the nearest centroid.
- 3. Update Step:** After all points have been assigned to clusters, calculate the mean of the points in each cluster and update the centroid to be the mean. This moves the centroid to the center of its cluster.
- 4. Repeat:** Repeat steps 2 and 3 until convergence. Convergence occurs when the centroids no longer change significantly or when a maximum number of iterations is reached.
- 5. Final Step:** Once convergence is reached, the algorithm outputs the final centroids and the cluster assignments for each data point.

Executable python code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
data = np.random.rand(1000, 2) * 100
km = KMeans(n_clusters=3, init="random")
km.fit(data)
centers = km.cluster_centers_
labels = km.labels_
print("Cluser centers: ", *centers)
# print("Cluser Labels: ", *labels)
colors = ["r", "g", "b"]
markers = ["+", "x", "*"]
for i in range(len(data)):
    plt.plot(data[i][0], data[i][1], color=colors[labels[i]],
marker=markers[labels[i]])
plt.scatter(centers[:, 0], centers[:, 1], marker="s", s=100,
linewidths=5)
plt.show()
```

Output:

```
Cluser centers:  [82.29904926  50.2243326 ] [31.71356124  22.45600779]
[31.52067957  77.620532  ]
```



Conclusion:

Hence, we implemented k-means clustering algorithm in python using google colab.

B.Sc.CSIT 7th Semester

CSC-420: Data Warehousing and Data Mining

Lab 02 :

Implementation of Apriori algorithm

Objective:

Write a Python program to utilize the Apriori algorithm on a retail dataset and identify significant association rules between items in customer transactions.

Required Theory:

The Apriori algorithm is a classical algorithm in data mining and machine learning used for association rule mining in transactional databases. It aims to find interesting relationships or associations among a set of items in large datasets. The most common application of the Apriori algorithm is in market basket analysis, where it helps identify associations between products that are frequently purchased together.

Algorithm Steps:

1. *Generating Candidate Itemsets:*

- The Apriori algorithm starts by generating candidate itemsets of length 1 (individual items) and then iteratively generates larger itemsets.
- New candidate itemsets are generated by joining pairs of frequent itemsets found in the previous iteration.

2. *Pruning Candidate Itemsets:*

- After generating candidate itemsets, the algorithm scans the dataset to count the support of each candidate itemset.
- Candidate itemsets that do not meet the minimum support threshold are pruned from further consideration.

- Once frequent itemsets are identified, association rules are generated from these itemsets.
- Association rules are generated by partitioning frequent itemsets into non-empty subsets and calculating support, confidence, and lift for each rule.
- Rules that meet the minimum confidence and lift thresholds are considered significant and are returned as the final output of the algorithm.

We are using a dataset of a retail store which contains 7501 total customer transactions (rows) in a CSV file. A snapshot of the dataset transaction is given below:

[illegible]

Executable python code:

```
!pip install apyori
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from apyori import apriori
path="/content/drive/MyDrive/DataSet/store_data.csv"
dataset=pd . read_csv (path)
dataset.head(None)
records = []
for i in range(0, 7500):
    test = []
    data = dataset.iloc[i]
    data = data.dropna()
    for j in range(0, len(data)):
        test.append(str(dataset.values[i, j]))
    records.append(test)
association_rules = apriori(
    records, min_support=0.005, min_confidence=0.2, min_lift=3,
min_length=2
)
association_results = list(association_rules)
for item in association_results:
    print(list(item[2][0][0]), '->', list(item[2][0][1]))
```

Output:

Association rules generated:

```
['mushroom cream sauce'] -> ['escalope']
['pasta'] -> ['escalope']
['herb & pepper'] -> ['ground beef']
['tomato sauce'] -> ['ground beef']
['whole wheat pasta'] -> ['olive oil']
['pasta'] -> ['shrimp']
['chocolate', 'frozen vegetables'] -> ['shrimp']
['spaghetti', 'frozen vegetables'] -> ['ground beef']
['shrimp', 'mineral water'] -> ['frozen vegetables']
['spaghetti', 'frozen vegetables'] -> ['olive oil']
['spaghetti', 'frozen vegetables'] -> ['shrimp']
['spaghetti', 'frozen vegetables'] -> ['tomatoes']
['spaghetti', 'grated cheese'] -> ['ground beef']
['herb & pepper', 'mineral water'] -> ['ground beef']
['herb & pepper', 'spaghetti'] -> ['ground beef']
['shrimp', 'ground beef'] -> ['spaghetti']
['milk', 'spaghetti'] -> ['olive oil']
['mineral water', 'soup'] -> ['olive oil']
['pancakes', 'spaghetti'] -> ['olive oil']
```

Conclusion:

Hence, we implemented Apriori algorithm in python using google collab.

B.Sc.CSIT 7th Semester

CSC-420: Data Warehousing and Data Mining

Lab 03 :

Implementation of ID3 decision tree algorithm

Objective:

Write a python program to predict diabetes using ID3 Decision Tree Classifier.

Required Theory:

The ID3 (Iterative Dichotomiser 3) algorithm is a classic and straightforward algorithm used for constructing decision trees. It was developed by Ross Quinlan in 1986 and is particularly popular for its simplicity and ease of understanding.

ID3 algorithm works as:

1. **Input Data:** ID3 algorithm starts with a dataset containing features and corresponding target labels.
2. **Feature Selection:** It selects the best attribute to split the data at each node based on a criterion called Information Gain. Information Gain measures how much entropy (uncertainty or randomness) is reduced in the dataset after splitting on a particular attribute.
3. **Tree Construction:** It recursively constructs the decision tree by selecting the best attribute to split the data at each node. This process continues until one of the stopping criteria is met, such as:
 - All instances at a node belong to the same class.
 - No more attributes are left to split on.
 - The tree reaches a maximum depth.

4. **Output:** The resulting decision tree is used for classification by following the decision paths from the root to the leaf nodes based on the values of the features of the input data.

Dataset Description:

We are using a dataset of a hospital which contains 768 total patients transactions (rows) in a CSV file. A snapshot of the dataset transaction is given below:

1	Pregnancy	Glucose	Blod Press	Skin Thikness	Insulin	BMI	DFP	Age	Diabetes
2	1	85	66	29	0	26.6	0.351	31	0
3	8	183	64	0	0	23.3	0.672	32	1
4	1	89	66	23	94	28.1	0.167	21	0
5	0	137	40	35	168	43.1	2.288	33	1
6	5	116	74	0	0	25.6	0.201	30	0
7	3	78	50	32	88	31	0.248	26	1
8	10	115	0	0	0	35.3	0.134	29	0
9	2	197	70	45	543	30.5	0.158	53	1
10	8	125	96	0	0	0	0.232	54	1
11	4	110	92	0	0	37.6	0.191	30	0
12	10	168	74	0	0	38	0.537	34	1
13	10	139	80	0	0	27.1	1.441	57	0
14	1	189	60	23	846	30.1	0.398	59	1
15	5	166	72	19	175	25.8	0.587	51	1
16	7	100	0	0	0	30	0.484	32	1
17	0	118	84	47	230	45.8	0.551	31	1
18	7	107	74	0	0	29.6	0.254	31	1
19	1	103	30	38	83	43.3	0.183	33	0
20	1	115	70	30	96	34.6	0.529	32	1
21	3	126	88	41	235	39.3	0.704	27	0
22	8	99	84	0	0	35.4	0.388	50	0
23	7	196	90	0	0	39.8	0.451	41	1
24	9	119	80	35	0	29	0.263	29	1
25	11	143	94	33	146	36.6	0.254	51	1
26	10	125	70	26	115	31.1	0.205	41	1
27	7	147	76	0	0	39.4	0.257	43	1
28	1	97	66	15	140	23.2	0.487	22	0
29	13	145	82	19	110	22.2	0.245	57	0
30	5	117	92	0	0	34.1	0.337	38	0
31	5	109	75	26	0	36	0.546	60	0

Executable python code:

```
!pip import pandas as pd
from sklearn import metrics
from sklearn.tree import DecisionTreeClassifier
path="/content/drive/MyDrive/DataSet/Diabetes.csv"
dataset = pd.read_csv(path)
print("Dataset Size: ", len(dataset))
split = int(len(dataset) * 0.7)
train, test = dataset.iloc[:split], dataset.iloc[split:]
p = train["Pregnancy"].values
g = train["Glucose"].values
bp = train["Blood Pressure"].values
st = train["Skin Thickness"].values
ins = train["Insulin"].values
bmi = train["BMI"].values
dpf = train["DFP"].values
a = train["Age"].values
d = train["Diabetes"].values
trainfeatures = zip(p, g, bp, st, ins, bmi, dpf, a)
traininput = list(trainfeatures)
# print(traininput)
model = DecisionTreeClassifier(criterion="entropy", max_depth=4)
model.fit(traininput, d)
p = test["Pregnancy"].values
g = test["Glucose"].values
bp = test["Blood Pressure"].values
st = test["Skin Thickness"].values
ins = test["Insulin"].values
bmi = test["BMI"].values
dpf = test["DFP"].values
a = test["Age"].values
d = test["Diabetes"].values
testfeatures = zip(p, g, bp, st, ins, bmi, dpf, a)
testinput = list(testfeatures)
predicted = model.predict(testinput)
# print('Actual Class:', *d)
# print('Predicted Class:', *predicted)
print("Confusion Matrix:")
print(metrics.confusion_matrix(d, predicted))
print("\nClassification Measures:")
print("Accuracy:", metrics.accuracy_score(d, predicted))
print("Recall:", metrics.recall_score(d, predicted))
print("Precision:", metrics.precision_score(d, predicted))
print("F1-score:", metrics.f1_score(d, predicted))
```

Output:

Association rules generated:

Dataset Size: 767

Confusion Matrix:

```
[[117  35]
 [ 17  62]]
```

Classification Measures:

Accuracy: 0.7748917748917749

Recall: 0.7848101265822784

Precision: 0.6391752577319587

F1-score: 0.7045454545454545

Conclusion:

Hence, we implemented ID3 decision tree algorithm in python using google colab.

B.Sc.CSIT 7th Semester

CSC-420: Data Warehousing and Data Mining

Lab 04 :

Implementation of DBSCAN clustering algorithm

Objective:

Write a python program to implement DBSCAN clustering algorithm.

Required Theory:

The DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a popular clustering algorithm used in machine learning and data mining. It's particularly useful for identifying clusters of arbitrary shapes in spatial data, and it's robust to outliers.

Here's how DBSCAN works:

1. Density-Based: DBSCAN defines clusters as areas in the data space where there are many data points concentrated together, separated by areas with few or no data points. It doesn't assume that clusters have a specific shape.

2. Parameters:

- Epsilon (ϵ): This is a distance threshold that defines the radius within which to search for neighboring points. It determines the minimum number of points required to form a dense region.

- MinPts: This parameter specifies the minimum number of points required to form a dense region. Any point with at least MinPts points within distance ϵ is considered a core point.

3. Core Points: A core point is a point that has at least MinPts neighboring points within distance ϵ . Core points lie deep within a cluster.

4. Border Points: Border points are not core points themselves but are within the ϵ neighborhood of a core point. They belong to the same cluster as the core point but lie on the edges.

5. Noise Points (Outliers): Points that are neither core points nor border points are considered noise points or outliers. They do not belong to any cluster.

6. Algorithm Steps:

- DBSCAN starts by randomly selecting a point from the dataset.
- It then finds all points in the ϵ -neighborhood of this point. If the number of points in this neighborhood is less than MinPts, the point is labeled as noise. Otherwise, it's labeled as a core point and assigned to a new cluster or an existing cluster.
- It continues this process recursively, expanding the cluster by adding core points and their reachable neighbors until no more points can be added.
- Once all points are either assigned to a cluster or labeled as noise, the algorithm terminates.

Executable python code:

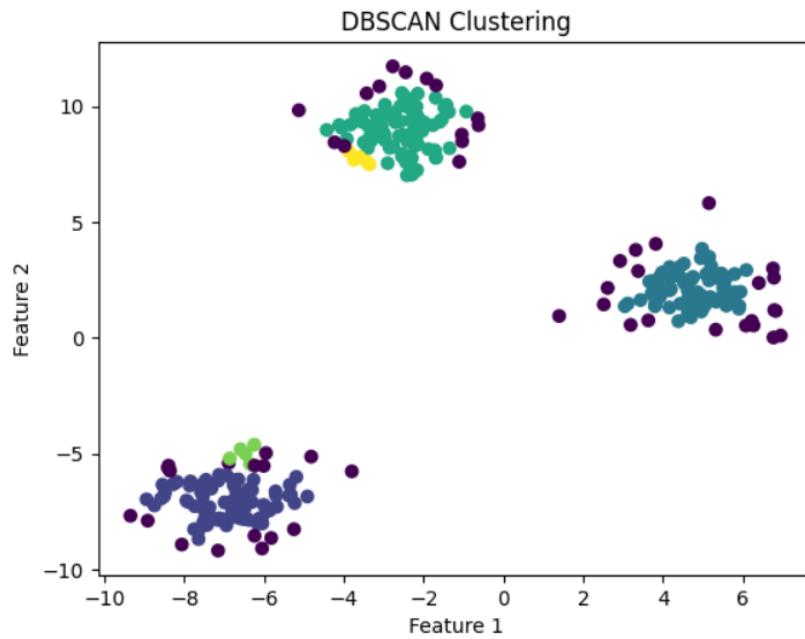
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_blobs

# Generate some random data for demonstration
data, labels = make_blobs(n_samples=300, centers=3, random_state=42)

# Use DBSCAN for clustering
dbscan = DBSCAN(eps=0.5, min_samples=5)
clusters = dbscan.fit_predict(data)

# Plot the results
plt.scatter(data[:, 0], data[:, 1], c=clusters, cmap='viridis')
plt.title('DBSCAN Clustering')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

Output:



Conclusion:

Hence, we implemented DBSCAN clustering algorithm in python using google collab.

5. Naive Bayes Classifier

1. Write a Python program to predict diabetes using Naive Bayes Classification.

```
import pandas as pd
from sklearn import metrics
from sklearn.naive_bayes import GaussianNB

dataset = pd.read_csv("Diabetes.csv")

print("Dataset Size: ", len(dataset))

split = int(len(dataset) * 0.7)
train, test = dataset.iloc[:split], dataset.iloc[split:]

p = train["Pragnency"].values
g = train["Glucose"].values
bp = train["Blod Pressure"].values
st = train["Skin Thikness"].values
ins = train["Insulin"].values
bmi = train["BMI"].values
dpf = train["DFP"].values
a = train["Age"].values
d = train["Diabetes"].values

trainfeatures = zip(p, g, bp, st, ins, bmi, dpf, a)
traininput = list(trainfeatures)
# print(traininput)

model = GaussianNB()
model.fit(traininput, d)

p = test["Pragnency"].values
g = test["Glucose"].values
bp = test["Blod Pressure"].values
st = test["Skin Thikness"].values
ins = test["Insulin"].values
bmi = test["BMI"].values
dpf = test["DFP"].values
a = test["Age"].values
d = test["Diabetes"].values

testfeatures = zip(p, g, bp, st, ins, bmi, dpf, a)
```

```
testinput = list(testfeatures)

predicted = model.predict(testinput)
# print('Actual Class:', *d)
# print('Predicted Class:', *predicted)

print("Confusion Matrix:")
print(metrics.confusion_matrix(d, predicted))

print("\nClassification Measures:")
print("Accuracy:", metrics.accuracy_score(d, predicted))
print("Recall:", metrics.recall_score(d, predicted))
print("Precision:", metrics.precision_score(d, predicted))
print("F1-score:", metrics.f1_score(d, predicted))

Dataset Size: 767
Confusion Matrix:
[[128  24]
 [ 30  49]]

Classification Measures:
Accuracy: 0.7662337662337663
Recall: 0.620253164556962
Precision: 0.6712328767123288
F1-score: 0.6447368421052632
```

6 Multilayer Perceptron

1. Write a Python program to predict breast cancer data using multilayer perceptron.
Compare the performance of both classifiers.

```
from sklearn import datasets
from keras.models import Sequential
from keras.layers import Dense
from sklearn import metrics
import numpy as np

cancer = datasets.load_breast_cancer()
x = cancer.data
y = cancer.target

split = int(len(x) * 0.7)
trainx, testx = x[:split], x[split:]
trainy, testy = y[:split], y[split:]

print("Number of features: ", len(cancer.feature_names))
# print("Features: ", *cancer.feature_names)
print("Number of classes: ", len(cancer.target_names))
print("Class Labels: ", cancer.target_names)

# Define the keras model
model = Sequential()
model.add(Dense(128, input_dim=30, activation="relu"))
model.add(Dense(128, activation="relu"))
model.add(Dense(128, activation="relu"))
model.add(Dense(1, activation="sigmoid"))

# Compile and fit the model
model.compile(loss="binary_crossentropy", optimizer="adam",
metrics=["accuracy"])
model.fit(trainx, trainy, epochs=200, batch_size=16, verbose=0)

# Make class predictions with the model
yp = model.predict(testx)

pred = []
for x in yp:
    pred.append(np.round(x))
pred = np.array(pred)
pred = pred.ravel()
pred = pred.astype(int)
```

```
# print("Actual Class: ", *testy)
# print("Predicted Class: ", *yp)

print("\nConfusion Matrix:")
print(metrics.confusion_matrix(testy, pred))

print("\nClassification Measures:")
print("Accuracy:", metrics.accuracy_score(testy, pred))
print("Recall:", metrics.recall_score(testy, pred))
print("Precision:", metrics.precision_score(testy, pred))
print("F1-score:", metrics.f1_score(testy, pred))

Number of features: 30
Number of classes: 2
Class Labels: ['malignant' 'benign']

Confusion Matrix:
[[ 36   3]
 [  2 130]]

Classification Measures:
Accuracy: 0.9707602339181286
Recall: 0.9848484848484849
Precision: 0.9774436090225563
F1-score: 0.981132075471698
```